In [1]:
```
import numpy as np
import matplotlib.pyplot as plt
```

In [2]:
```
X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1],
    [0.2, 0.3],
    [0.8, 0.9],
    [0.1, 0.8],
    [0.9, 0.2]
], dtype=float)

y = np.array([0, 0, 0, 1, 0, 1, 0, 0]).reshape(-1, 1)

print("Input Data:\n", X)
print("Target Labels:\n", y.ravel())
```

```
Input Data:
 [[0.  0. ]
 [0.  1. ]
 [1.  0. ]
 [1.  1. ]
 [0.2 0.3]
 [0.8 0.9]
 [0.1 0.8]
 [0.9 0.2]]
Target Labels:
 [0 0 0 1 0 1 0 0]
```

In [3]:
```
def sigmoid(z):
    return 1 / (1 + np.exp(-np.clip(z, -500, 500)))
```

In [4]:
```
np.random.seed(42)

weights = np.random.randn(2, 1) * 0.1
bias = 0.0

learning_rate = 0.5
epochs = 10000
m = len(X)
```

In [5]:
```
for epoch in range(epochs):
    # Forward propagation
    z = np.dot(X, weights) + bias
    predictions = sigmoid(z)

    # Calculate gradients
    dw = np.dot(X.T, predictions - y) / m
    db = np.mean(predictions - y)

    # Update weights and bias
    weights -= learning_rate * dw
```

```
    bias -= learning_rate * db

print("Training completed.")
print("Weights:\n", weights)
print("Bias:", bias)
```

```
Training completed.
Weights:
 [[11.94733021]
 [13.34222279]]
Bias: -17.798277258872762
```

In [6]:
```
probabilities = sigmoid(np.dot(X, weights) + bias)
predicted_classes = (probabilities >= 0.5).astype(int)

print("Actual Classes:", y.ravel())
print("Predicted Classes:", predicted_classes.ravel())
```

```
Actual Classes: [0 0 0 1 0 1 0 0]
Predicted Classes: [0 0 0 1 0 1 0 0]
```

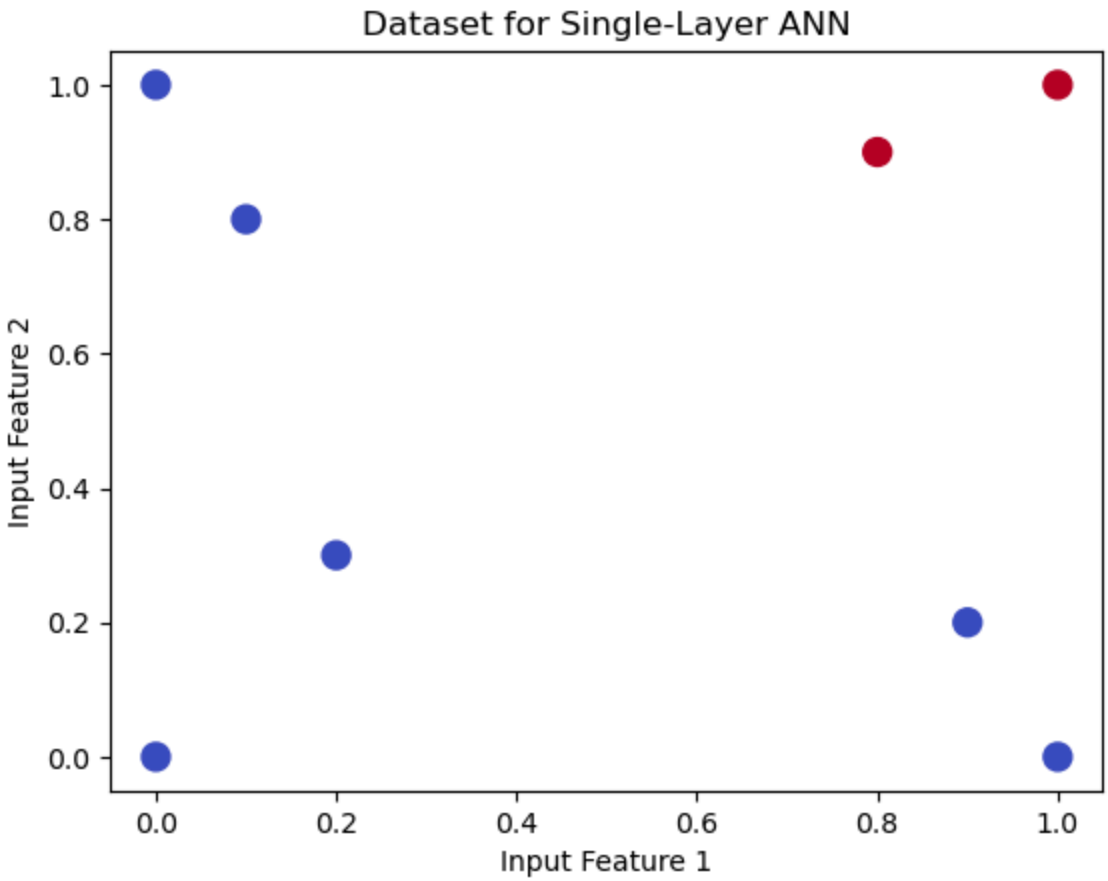
In [7]:
```
accuracy = np.mean(predicted_classes == y) * 100

print("Model Accuracy:", round(accuracy, 2), "%")
```

```
Model Accuracy: 100.0 %
```

In [8]:
```
plt.scatter(
    X[:, 0], X[:, 1],
    c=y.ravel(),
    cmap="coolwarm",
    s=100
)

plt.xlabel("Input Feature 1")
plt.ylabel("Input Feature 2")
plt.title("Dataset for Single-Layer ANN")
plt.show()
```



In [9]:
```
new_data = np.array([[0.7, 0.8]])

new_probability = sigmoid(np.dot(new_data, weights) + bias)
new_class = (new_probability >= 0.5).astype(int)

print("Predicted Class:", new_class[0, 0])
```

```
Predicted Class: 1
```

In [ ]:
```

```